MODUL PEMROGRAMAN 1

Disusun oleh :

Dosen 1

Dosen 2

Instruktur 1

.....

LP2M Politeknik Manufaktur Astra

JAKARTA

MODUL Pemrograman 1

Penulis :

Dosen Pengampu

Instruktur

Penerbit :

LP2M Politeknik Manufaktur Astra

Jl. Gaya Motor Raya No.8 Sunter II Jakarta 14330

Telepon: (021) 6519555 Fax: (021) 6519821

Email: sekretariat@polman.astra.ac.id

KATA PENGANTAR

Modul praktikum ini dibuat sebagai pedoman mahasiswa dalam melakukan kegiatan praktikum mata kuliah ……. di Politeknik Astra. Modul praktikum ini terdiri dari …….. topik praktikum yang membahas tentang ……. Modul praktikum ini diharapkan dapat membantu mahasiswa/i dalam melaksanakan praktikum dengan lebih baik, terarah, dan terencana. Pada setiap topik telah ditetapkan tujuan pelaksanaan praktikum dan semua kegiatan yang harus dilakukan oleh mahasiswa/i serta teori singkat untuk memperdalam pemahaman mahasiswa/i mengenai materi yang dibahas.

Penyusun menyakini bahwa dalam pembuatan Modul Praktikum mata kuliah ……... ini masih jauh dari sempurna. Oleh karena itu penyusun mengharapkan kritik dan saran yang membangun guna penyempurnaan modul praktikum ini dimasa yang akan datang. Akhir kata, penyusun mengucapkan banyak terima kasih kepada semua pihak yang telah membantu baik secara langsung maupun tidak langsung.

Jakarta, …….

Penulis

DAFTAR ISI

KATA PENGANTAR ii

DAFTAR ISI iii

M04 Struktur Algoritma Sequence (Runtutan) dan Selection (Pemilihan) 1

1. TUJUAN 1
2. DURASI WAKTU 1
3. DASAR TEORI 1
4. PERCOBAAN 12
5. LATIHAN 18

M05 PENGULANGAN / *REPETITION*.

1. TUJUAN

CPMK :

1. Mahasiswa mampu menjelaskan dan mengimplementasikan struktur algoritma *repetition* (pengulangan).

Sub-CPMK :

1. Mahasiswa mampu membuat algoritma menggunakan struktur algoritma *repetition* (pengulangan) dan mentranslasikan kedalam bahasa pemrograman C.

2. DURASI WAKTU

2 pertemuan x 2 jam

3. DASAR TEORI

Adapun karakteristik penggunaan struktur algoritma pengulangan adalah sebagai berikut :

- Pengulangan / *repetition* harus berhenti.
- *Programmer* harus mengetahui pengulangan akan berhenti sebelum program dijalankan atau saat merancang program. Karena itu saat menggunakan struktur kontrol pengulangan *programmer* harus merancang *initialization* dan *termination*.
    - *Initialization* merupakan instruksi yang dieskekusi sebelum pengulangan dilakukan dan nantinya akan digunakan untuk terminasi atau kondisi yang menyebabkan proses pengulangan berhenti.
    - *Termination* merupakan akhir dari pengulangan, akan dieksekusi ketika pengulangan sudah berhenti atau ketika kondisi yang menyebabkan sebuah pengulangan sudah tidak terpenuhi.

- Konsep pengulangan disarankan menggunakan hanya satu pintu masuk dan satu pintu keluar. Karena itu tidak disarankan menggunakan lebih dari satu pintu keluar, yaitu penggunaan *break* saat proses pengulangan.

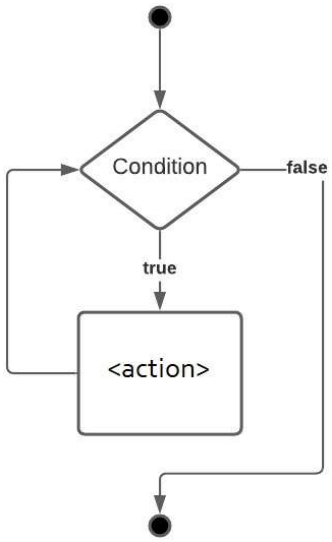
Jenis Struktur Algoritma Pengulangan

1. While – Loop

Karakteristik pengulangan **while – loop** adalah :

- Pengecekan kondisi pengulangan dilakukan di awal sebelum instruksi pengulangan dilakukan,
- Proses pengulangan dilakukan selama kondisi TRUE
- Proses akan keluar dari *loop* ketika kondisi FALSE

Perhatikan flowchart pengulangan menggunakan while-loop dibawah ini



Keterangan **:**

Condition merupakan ekspresi yang bernilai Boolean (TRUE atau FLASE)

Action dapat berisi 1 atau lebih instruksi / statement yang dieksekusi selama kondisi **TRUE**

Pseudocode dan bahasa pemrograman C

Pseudocode	Bahasa C

`while <condition> do`
`  <action>`
`endwhile`
	`while (condition) {`
	`  ... <action>`
	`}`

Contoh penggunaan bentuk while – loop :

Penulisan dalam Bahasa Pseudocode dan Bahasa C

```
program contohwhileloop
{tampilkan pada layar halo ke-i mulai dari i = 1 sampai dengan i = 10}

kamus
    i : integer

algoritma

    i <- 1  {initialization}

    while (i <= 10) do
        write(halo ke-,i)
        i <- i + 1
    endwhile  {termination : loop akan berhenti ketika i > 10}

endprogram
```

```c
/*tampilkan pada layar halo ke-i mulai dari i = 1 sampai dengan i = 10*/

#include <stdio.h>


void main(){
    int i;

    i = 1;  /*initialization*/

    while (i <= 10) {
        printf("halo ke-%d\n",i);
        i++;
    }  /*termination : loop akan berhenti ketika i > 10*/

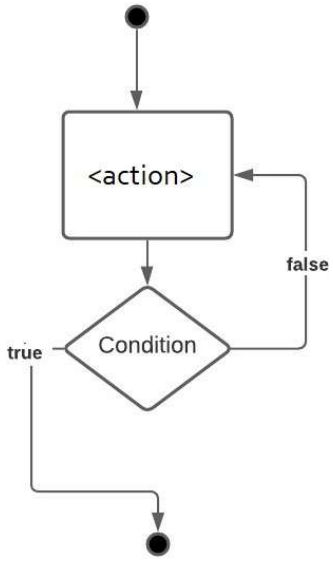
}
```

2. repeat – until

Karakteristik pengulangan **repeat – until** adalah :

- Instruksi **dieksekusi minimal 1x**, karena pemeriksaan kondisi dilakukan di akhir instruksi pengulangan dilakukan.
- Proses pengulangan dilakukan selama kondisi FALSE, hal ini berbanding terbalik dengan pengulangan menggunakan while-loop
- Proses akan keluar dari *loop* ketika kondisi TRUE

Perhatikan flowchart pengulangan menggunakan repeat-until dibawah ini



Keterangan **:**

Action dapat berisi 1 atau lebih instruksi / statement yang dieksekusi **minimal 1 kali** dan akan diulangi selama kondisi FALSE

Condition merupakan ekspresi yang bernilai Boolean (TRUE atau FLASE)

Pseudocode dan bahasa pemrograman C

Pseudocode	Bahasa C
repeat <action> until <condition>	**Konsep repeat – until tidak dikenali dalam bahasa C.** Meskipun begitu karakteristik repeat-until dimana instruksi pengulangan dieksekusi **minimal 1x** (pengecekan dilakukan di akhir instruksi pengulangan) dapat menggunakan syntax do-while do { ...<action> }while(condition);

Contoh penggunaan bentuk repeat – until :

Penulisan dalam bahasa Psedudocode

```
program contohrepeatuntil
{tampilkan pada layar halo ke-i mulai dari i = 1 sampai dengan i = 10}

kamus
    i : integer

algoritma

    i <- 1 {initialization}

    repeat
        write(halo ke-,i)
        i <- i + 1
    until( i > 10 ) {termination : loop akan berhenti ketika i > 10}

endprogram
```

Penulisan dalam bahasa pemrograman C

(*menggunakan do-while)

```
/*tampilkan pada layar halo ke-i mulai dari i = 1 sampai dengan i = 10*/
#include <stdio.h>

void main(){
    int i;

    i = 1;  /*initialization*/
    do {
        printf("%d\n",i);
        i++;
    }while(i <= 10);  /*termination : loop akan berhenti ketika i > 10*/

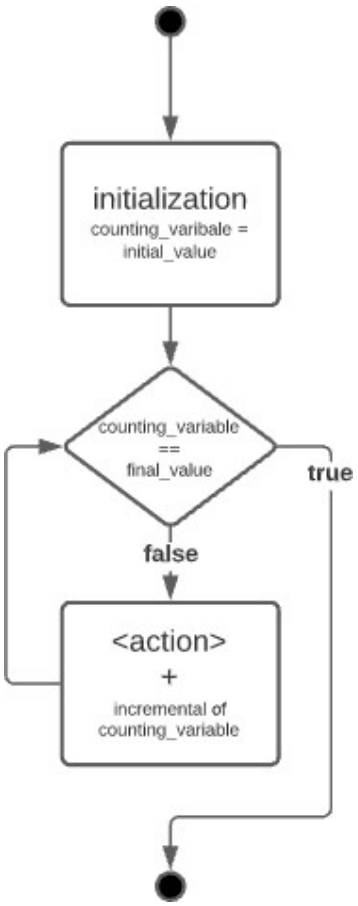
}
```

3. for

Karakteristik pengulangan **for** adalah :

- Digunakan saat instruksi pengulangan yang akan dilakukan sudah diketahui sebanyak berapa kali.
- Terminasi dilakukan ketika instruksi pengulangan sudah dilakukan sebanyak yang diinginkan. Karena itu kondisi untuk terminasi berbentuk bilangan bulat.
- Memiliki 3 pernyataan penting yaitu : counting_variable, initial_value dan final_value.
    - counting_variable digunakan untuk menghitung banyaknya proses pengulangan dilakukan, yang secara otomatis akan selalu bertambah setiap instruksi pengulangan telah dilakukan
    - initial_value digunakan sebagai *initialization* atau nilai awal yang diisikan pada counting_variable
    - final_value digunakan sebagai *termination* dimana akan menjadi terminasi ketika nilai counting_variable telah mencapai final_value.

Perhatikan flowchart pengulangan menggunakan for dibawah ini



Keterangan **:**

Initialization dimana counting_variable diisi dengan initial_value

Termination akan terjadi kerika counting_variable sudah mencapai final_value

Action akan dikerjakan selama kondisi terminasi false dan setelah action dilakukan nilai counting_variable akan *incremental

*dapat diset untuk decremental

Pseudocode dan bahasa pemrograman C

Pseudocode	Bahasa C

for counting_variable <- initial_value to final_value do
  <action>
endfor
	for(initialization; termination ; incremental){
	  ...<action>
	}

for counting_variable <- initial_value downto final_value do
  <action>
endfor
	for(initialization ; termination ; decremental){
	  ...<action>
	}

Contoh penggunaan bentuk **for –to** :

Penulisan dalam bahasa Psedudocode

```
program contohfor
{tampilkan pada layar halo ke-i mulai dari i = 1 sampai dengan i = 10}

kamus
    i : integer

algoritma


    for i <- 1 to 10 do  {initialization}
        write(halo ke-,i)
    endfor  {termination : loop akan berhenti ketika i sudah mencapai nilai 10}

endprogram
```

Penulisan dalam bahasa pemrograman C

```
/*tampilkan pada layar halo ke-i mulai dari i = 1 sampai dengan i = 10*/
#include <stdio.h>

int main () {

   int i;


   for( i = 1; i < 11; i = i + 1 ){
      printf("halo ke-%d\n", i);
   }

   return 0;
}
```

Contoh penggunaan bentuk **for –downto** :

Penulisan dalam bahasa Psedudocode

```
program contohfordownto
{tampilkan pada layar halo ke-i mulai dari i = 10 sampai dengan i = 1}

kamus
    i : integer

algoritma


    for i <- 10 downto 1 do  {initialization}
        write(halo ke-,i)
    endfor  {termination : loop akan berhenti ketika i sudah mencapai nilai 1}

endprogram
```

Penulisan dalam bahasa pemrograman C

```c
#include <stdio.h>

int main () {

    int i;


    for( i = 10; i  > 0; i -- ){
       printf("halo ke-%d\n", i);
    }

    return 0;
}
```

4. PERCOBAAN

Percobaan 1. While - loop

Buatlah program untuk selalu menerima input pengguna selama input pengguna tidak valid. Input valid ketika yang diinputkan adalah bilangan negatif. Gunakan struktur pengulangan while-loop

Pseudocode

```
program percobaanSoal1
{program akan menerima input selama input tidak valid.}
{dinyatakan valid ketika input adalah bilangan positif}

Kamus
a : integer

algoritma

write("masukan angka : ")
read(a)
while a < 0 do
    write("angka yang tidak valid!, masukan angka baru : ")
    read(a)
endwhile

write("terima kasih")
endprogram
```

Kode Program Bahasa C

```
#include <stdio.h>

void main(){

    int a;

    printf("masukan angka ");
    scanf("%d",&a);

    while (a < 0) {
        printf("angka tidak valid!,masukan angka baru :  ");
        scanf("%d",&a);

    }

    printf("\nterima kasih");
}
```

1. Tulis ulang program C diatas.

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int a;
    // Algorithm
    printf("Masukkan angka: ");
    scanf("%d", &a);
    while(a<0) {
        printf("Angka tidak valid!, masukan angka baru : ");
        scanf("%d", &a);
    }
    printf("\nTerimakasih");
}
```

2. Tentukan initialization and termination loop pada program diatas

   Initialization pada program diatas adalah scanf("%d",&a); pada line 8

   Termination pada program diatas terjadi ketika while(a<0) kondisi terdapat pada line 10, *Action* pengulangan terdapat pada line berapa saja? Pada line 10 - 12 dan proses apa yang dilakukan saat proses pengulangan adalah proses di dalam loop adalah menampilkan pesan "Angka tidak valid!" lalu meminta input ulang (printf + scanf).

3. Modifikasi bahasa C sehingga dapat menampilkan banyaknya input tidak valid.

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int a;
    int n = 0;
    // Algorithm
    printf("Masukkan angka: ");
    scanf("%d", &a);
    while(a<0) {
        n++;
        printf("Angka tidak valid!, masukan angka baru : ");
        scanf("%d", &a);
    }
    printf("Angka tidak valid : %d",n);
}
```

4.

5.

Contoh masukan dan keluaran sebagai berikut :

Masukan	Keluaran
-2 -9 -8 -1 -2 -7 4	Angka tidak valid : 6
3	Angka tidak valid : 0
-3 -1 -5 0	Angka tidak valid : 3

Percobaan 2. Repeat -until

Buatlah program untuk meminta pengguna untuk menebak angka rahasia (1 s.d 10) Kesempatan untuk menebak angka hanya 3x. Gunakan struktur pengulangan repeat-until. Angka rahasia dapat dibuat sebagai konstanta pada program.

Pseudocode

```
program percobaan2

Kamus
const angkarahasia : integer = 10
tebakan : integer
i : integer

Algoritma

i <- 1
repeat
    write("tebakan ke-", i);
    read(tebakan);
    i <- i + 1
until( i > 3 or tebakan == angkarahasia )

endprogram
```

Kode Program Bahasa C

```
#include <stdio.h>

int main(){
    const int angkarahasia = 10;
    int tebakan, i;

    i = 1;
    do {
        printf("\nTebakan ke-%d : ", i);
        scanf("%d",&tebakan);
        i++;
    }while(i <= 3 and tebakan != angkarahasia);

}
```

1. Tulis ulang program C diatas.

```c
#include <stdio.h>
int main(void) {
    // Declaration
    const int angkarahasia = 10;
    int tebakan, i;
    // Algorithm
    i=1;
    do {
        printf("\nTebakan ke-%d: ",i);
        scanf("%d", &tebakan);
        i++;
    } while (i <= 3 && tebakan != angkarahasia);
}
```

2. Tentukan initialization and termination loop pada program diatas

   Initialization pada program diatas adalah i=1 pada line 7

   Termination pada program diatas terjadi ketika i sudah 3 atau tebakan sama dengan angkarahasia, kondisi terdapat pada line 12 *Action* pengulangan terdapat pada line berapa saja? Pada line 8-12 dan proses apa yang dilakukan saat proses pengulangan, dilakukan proses memeriksa I apakah sudah 3 dan mengecek tebakan tidak sama dengan angkarahasia

3. Modifikasi bahasa C sehingga dapat menampilkan apakah user menang atau kalah. User menang ketika tebakan sesuai. Dapat mengkombinasikan dengan struktur percabangan

```c
#include <stdio.h>
int main(void) {
    // Declaration
    const int angkarahasia = 10;
    int tebakan, i;
    // Algorithm
    i=1;
    do {
        printf("\nTebakan ke-%d: ",i);
        scanf("%d", &tebakan);
        i++;
    } while (i <= 3 && tebakan != angkarahasia);
    if (tebakan == angkarahasia) {
        printf("\nAnda Menang!");
    } else {
        printf("\nAnda Kalah!");
```

```
    }
}
```

Contoh masukan dan keluaran sebagai berikut **(angka rahasia adalah 10)** :

Masukan	Keluaran
Tebakan ke-1 : 3 Tebakan ke-2 : 9 Tebakan ke-3 : 2	Anda Kalah!
Tebakan ke-1 : 6 Tebakan ke-2 : 10	Anda Menang!
Tebakan ke-1 : 3 Tebakan ke-2 : 4 Tebakan ke-3 : 10	Anda Menang!

Percobaan 3. For

Buatlah sebuah program untuk mencari nilai sebuah fungsi deret aritmatika berikut, mengginakan fungsi perngulangan for.

$$(x) = \sum_{i=1}^{n=x} i = 1 + 2 + \cdots + n$$

Pseudocode

```
1   program percobaan3
2
3   kamus :
4
5       i : integer
6       x : integer
7       fx : integer
8
9   program
10
11      read(x)
12      fx <- 0
13      i <- 0
14      for i to x do
15         fx = fx + i;
16      endfor
17
18      printf("\nnilai f(x) = %d", fx);
19
20  endprogram
```

Kode Program Bahasa C

```
1   #include <stdio.h>
2
3   int main () {
4
5       int i;
6       int x;
7       int fx = 0;
8
9       scanf("%d",&x);
10
11      for( i = 1; i <= x ; i++ ){
12         fx = fx + i;
13      }
14      printf("\nnilai f(x) = %d", fx);
15  }
```

1. Tulis ulang program C diatas.

```c
#include <stdio.h>

int main(void) {
    // Declaration
    int i;
    int x;
    int fx=0;
    // Algorithm
    scanf("%d", &x);
    for (i=1; i<=x; i++) {
        fx = fx + i;
    }
    printf("\nnilai f(x)%d", fx);
}
```

2. Tentukan initialization and termination loop pada program diatas

   Initialization pada program diatas adalah fx=0 dan scanf("%d",&x); pada line 7 dan 9

   Termination pada program diatas terjadi ketika i > x , dan kondisi terdapat pada line 11, *Action* pengulangan terdapat pada line berapa saja? Pada line 11 dan proses apa yang dilakukan saat proses pengulangan jelaskan! Proses pengulangan: fx dijumlahkan dengan i (fx = fx + i), setelah itu i otomatis dinaikkan 1, diulang selama i <= x.

3. Modifikasi bahasa C untuk menyelesaikan persamaan notasi sigma pada deret artimatika berikut. Dimana nilai x adalah bilangan bulat.

$$f(x) = \sum_{i=1}^{n=x} 2\,i \; = (2 \times 1) + (2 \times 2) + \cdots + (2 \times n)$$

```c
#include <stdio.h>

int main(void) {
    // Declaration
    int i;
    int x;
    int fx = 0;
    // Algorithm
    scanf("%d", &x);
    for (i = 1; i <= x; i++) {
        fx = fx + 2*i;
    }
    printf("\nnilai f(x) = %d\n", fx);
    return 0;
}
```

Contoh masukan dan keluaran sebagai berikut :

Masukan	Keluaran	Keterangan
5	30	(2x1) + (2x2) + (2x3) + (2x4) + (2x5) = 30
7	56	(2x1) + (2x2) + (2x3) + (2x4) + (2x5) + (2x6) + (2x7) = 56

5. LATIHAN

Latihan T1.

Buatlah dalam bentuk algoritm notasi pseudocode lalu terjemahkan ke dalam kode program C.

1. Buatlah program factorial. Menggunakan **for - to**

```
program factorial
{menghitung n! untuk n bilangan bulat positif.
Perhatikan n=0 dimana nilai faktorial dari 0 adalah 1, 0! = 1}
```

Contoh :

Masukan	Keluaran	Keterangan
5	120	5! = 5 x 4 x 3 x 2 x 1
1	1	1! = 1 x 1

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i;
    int faktorial = 1;
    // Algorithm
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        faktorial = faktorial * i;
    }
    printf("%d\n", faktorial);
    return 0;
}
```

2. Buatlah program segitiga.

```
program segitiga
{menggambar segitiga dari * dengan ketinggian N, dimana baris ke-i berisi * sebanyak i.
Nilai dari N diperoleh dari input console}
```

Contoh :

Masukan	Keluaran
5	* ** *** **** *****
3	* ** ***

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i, j;
    // Algorithm
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

3. Buatlah program untuk mencari nilai minimum.

```
program NilaiMinimum
{mencari terkecil sebanyak N angka, dimana banyaknya N diperoleh
dari input console. }
```

Masukan : Berupa beberapa baris bilangan bulat. Baris pertama (N) adalah banyaknya input yang akan dilakukan oleh pengguna, lalu baris berikutnya adalah angka sebanyak N yang perlu dicari nilai terkecilnya.
Keluaran : Berupa bilangan bulat terkecil dari N angka yang diinputkan pengguna.

Contoh :

Masukan	Keluaran
5 -3 2 5 1 -9	Nilai terkecil adalah = -9
4 9 7 6 20	Nilai terkecil adalah = 6

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i, x, min;
    // Algorithm
    scanf("%d", &n);
    scanf("%d", &x);
    min = x;
    for (i = 2; i <= n; i++) {
        scanf("%d", &x);
        if (x < min) {
```

```
            min = x;
        }
    }
    printf("Nilai terkecil adalah = %d\n", min);
    return 0;
}
```

4. Buatlah program untuk menjumlahkan N kali bilangan ganjil pertama .

```
PROGRAM CountOddSeries
{ Menghitung jumlah N buah bilangan ganjil pertama (1 + 3 + 5 + ...).
  N dibaca dari keyboard. N adalah bilangan bulat non-negatif. }
```

Masukan : N adalah bilangan bulat positif yang menyatakan banyaknya bilangan ganjil yang dijumlahkan

Keluaran : Berupa total dari penjumlahan bilangan ganjil pertama sebanyak N

Contoh :

Masukan	Keluaran	Keterangan
3	9	Nilai 9 diperoleh dari 1 + 3 + 5
2	4	Nilai 9 diperoleh dari 1 + 3
5	25	Nilai 25 diperoleh dari 1+3+5+7+9

```
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i;
    int jumlah = 0;
    // Algorithm
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        jumlah = jumlah + (2*i - 1);
    }
    printf("%d\n", jumlah);
    return 0;
}
```

Latihan T2.

Buatlah dalam bentuk algoritm notasi pseudocode lalu terjemahkan ke dalam kode program C.

1. Buatlah program factorial. Menggunakan **for - down to** atau pengulangan yang menggunakan **for** tetapi counting_variablenya decremental bukan incremental

```
program factorialVer2
{menghitung n! untuk n bilangan bulat positif.
Perhatikan n=0 dimana nilai faktorial dari 0 adalah 1, 0! = 1}
```

Contoh :

Masukan	Keluaran	Keterangan
5	120	5! = 5 x 4 x 3 x 2 x 1
1	1	1! = 1 x 1

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i;
    int faktorial = 1;
    // Algorithm
    scanf("%d", &n);
    for (i = n; i >= 1; i--) {
        faktorial = faktorial * i;
    }
    printf("%d\n", faktorial);
    return 0;
}
```

2. Buatlah program segitiga versi 2

```
program segitigaVer2
{menggambar segitiga dari * dengan ketinggian N + (N-1),
Nilai dari N diperoleh dari input console}
```

Contoh :

Masukan	Keluaran
3	* ** *** ** *
2	* ** *

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i, j;

    // Algorithm
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    for (i = n - 1; i >= 1; i--) {
        for (j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

3. Buatlah program untuk mencari nilai maksimum dan minimum

```
program NilaiMaxMin
{mencari terkecil dan terbesar sebanyak N angka, dimana banyaknya N diperoleh
dari input console. }
```

Masukan : Berupa beberapa baris bilangan bulat. Baris pertama (N) adalah banyaknya input yang akan dilakukan oleh pengguna, lalu baris berikutnya adalah angka sebanyak N yang perlu dicari nilai terkecilnya dan terbesarnya.
Keluaran : Berupa bilangan bulat terkecil dan bilangan bulat terbesar dari N angka yang diinputkan pengguna.

Contoh :

Masukan	Keluaran
5 -3 2 5 1 -9	Nilai terkecil adalah = -9 Nilai terbesar adalah = 5
4 9 7 6 20	Nilai terkecil adalah = 6 Nilai terbesar adalah = 5

```c
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i, x, min, maks;
    // Algorithm
    scanf("%d", &n);
    scanf("%d", &x);
    min = x;
```

```
    maks = x;
    for (i = 2; i <= n; i++) {
        scanf("%d", &x);
        if (x < min) {
            min = x;
        }
        if (x > maks) {
            maks = x;
        }
    }
    printf("Nilai terkecil adalah = %d\n", min);
    printf("Nilai terbesar adalah = %d\n", maks);
    return 0;
}
```

4. Buatlah program untuk menjumlahkan semua bilangan ganjil 1 s.d N.

```
program CountOddSeriesVer2
{Menghitung jumlah N bilangan ganjil dari 1 sampai N (1 + 3 + 5 + ... + N).
 N dibaca dari keyboard. N adalah bilangan bulat non-negatif ganjil.
 Perhatikan perbedaannya dengan soal nomor 4 T1.
```

Masukan : N adalah bilangan ganjil

Keluaran : Berupa total dari penjumlahan bilangan ganjil pertama sampai dengan bilangan N

Contoh :

Masukan	Keluaran	Keterangan
3	4	Nilai 4 diperoleh dari 1 + 3
9	25	Nilai 25 diperoleh dari 1+3+5+7+9
5	9	Nilai 9 diperoleh dari 1+3+5

```
#include <stdio.h>
int main(void) {
    // Declaration
    int n, i;
```

```
    int jumlah = 0;
    // Algorithm
    scanf("%d", &n);
    for (i = 1; i <= n; i = i + 2) {
        jumlah = jumlah + i;
    }
    printf("%d\n", jumlah);
    return 0;
}
```